

Analyzing the Impact of Outliers on Linear Regression and Gradient-Based Optimization

항공전자정보공학부

Donghyeon Kim

ID: 2019124018

Korea Aerospace University

Abstract

We investigate the influence of outlier samples on linear regression and propose methods to enhance model robustness. Starting with synthetic data generated from known weight vectors, we compare analytical solutions with gradient descent under varying learning rates and initialization strategies. We extend this to a mixed-label setting, where a minority of samples follow an alternative weight vector, simulating unknown outliers. Our results show that performance degrades as outlier proportion increases, but tuned gradient descent can mitigate this effect. We demonstrate that incorporating data contamination awareness into optimization significantly reduces estimation errors and improves reliability in noisy environments.

1. Introduction

Linear regression is a foundational technique in both machine learning and statistics, widely used to model the relationship between input variables and a continuous target. Under ideal conditions, where data is clean and aligns well with the model assumptions, linear regression yields accurate and efficient results. However, real-world datasets are often contaminated with outliers samples that significantly deviate from the true underlying distribution due to factors such as measurement noise, labeling errors, or heterogeneous data sources. These outliers can severely impact model performance, leading to biased parameter estimates and reduced generalization ability.

This study examines how outlier contamination affects regression performance. We begin by generating synthetic datasets using known weight vectors, and evaluate both analytical solutions and iterative gradient descent methods under varying

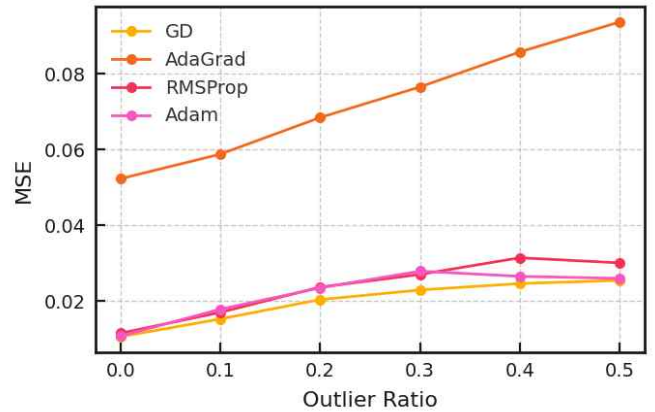


Figure 1. Mean squared error (MSE) after 1000 training epochs across varying outlier ratios. As the proportion of outliers increases, all optimizers show performance degradation to varying degrees. Among them, Adam demonstrates the most stable performance under increasing contamination,

conditions, such as learning rate schemes and initialization strategies. We then extend the setting to a more challenging mixed distribution scenario, where a subset of samples is generated using an alternative weight vector. This simulates the presence of undetectable outliers in practice.

Through systematic experiments, we analyze the influence of key factors such as contamination ratio, batch size, and optimization hyperparameters on model robustness. Our findings provide practical insights into improving regression reliability in noisy or structurally mixed data environments, a frequent challenge in real world applications.

2. Related work

This work examines how regression behaves when training data includes outliers. We analyze regression performance is affected by adaptive learning rate, initialization, and the proportion of contaminated

samples. Using synthetic datasets with controlled outlier ratios, we compare the accuracy of solutions obtained through analytical and iterative methods.

The goal is to understand how optimization settings influence robustness in noisy environments and to identify strategies that improve estimation accuracy under data contamination.

Linear regression is a foundational technique in supervised learning that models the relationship between an input vector $x \in R^D$ and a continuous output $y \in R$ through a linear mapping:

$$y = x^T w$$

where $x \in R^D$ is the parameter vector to be estimated. Given a dataset $\{(x_i, y_i)\}$, the model parameters are typically learned by minimizing the mean squared error (MSE):

$$\min \frac{1}{N} \sum_{i=1}^N (y_i - x_i^T w)^2$$

This problem admits a closed form solution using the normal equation:

$$w^* = (X^T X)^{-1} X^T y$$

Alternatively, the same objective can be optimized iteratively using gradient based methods such as stochastic gradient descent (SGD).

Gradient descent (GD) is a first order iterative optimization algorithm commonly used to minimize differentiable loss functions. In the context of linear regression, it aims to find the optimal weight vector w that minimizes the mean squared error between the predicted and actual outputs. Starting from an initial estimate w_0 , the weights are updated iteratively as:

$$w_{t+1} = w_t - \eta \nabla L(w_t)$$

where $\eta > 0$ is the learning rate and $\nabla L(w_t)$ is the gradient of the loss function at step t . For linear regression with MSE loss, the gradient is given by:

$$\nabla L(w) = -\frac{2}{N} X^T (y - Xw)$$

Stochastic Gradient Descent (SGD) is a variant of gradient descent that updates model parameters using only a single data point (or a small mini-batch) at each iteration, rather than the full dataset. This approach significantly reduces the computational cost per update and allows for faster convergence in large-scale problems. To mitigate instability in SGD, several adaptive optimizers such as AdaGrad, RMSProp, and Adam have been proposed.

AdaGrad (Adaptive Gradient) is an adaptive optimization algorithm designed to improve the stability and convergence of stochastic gradient descent, particularly in sparse or noisy settings. The key idea

is to scale the learning rate for each parameter based on the cumulative sum of its past squared gradients. Given a loss function $L(w)$, the update rule for parameter w_j at time t is:

$$w_j^{(t+1)} = w_j^{(t)} - \frac{\eta}{\sqrt{G_j^{(t)} + \epsilon}} \cdot g_j^{(t)}$$

where $g_j^{(t)}$ is the gradient of the loss with respect to w_j at iteration t , $G_j^{(t)} = \sum_{\tau=1}^t (g_j^{(\tau)})^2$, and ϵ is a small constant to prevent division by zero.

RMSProp (Root Mean Square Propagation) is an adaptive optimization algorithm that modifies AdaGrad by using an exponentially decaying average of past squared gradients, instead of their cumulative sum. This prevents the learning rate from shrinking too quickly and allows the model to continue learning even in long training runs. The update rule for parameter w_j at iteration t is given by:

$$s_j^{(t)} = \rho \cdot s_j^{(t-1)} + (1-\rho) \cdot (g_j^{(t)})^2, \\ w_j^{(t+1)} = w_j^{(t)} - \frac{\eta}{\sqrt{s_j^{(t)} + \epsilon}} \cdot g_j^{(t)}$$

where $s_j^{(t)}$ is the running average of squared gradients, $g_j^{(t)}$ is the current gradient, $\rho \in [0,1]$ is the decay rate.

Adam (Adaptive Moment Estimation) is a widely used adaptive optimization algorithm that combines the advantages of both AdaGrad and RMSProp. It maintains exponentially decaying averages of both the first moment (mean) and the second moment (uncentered variance) of the gradients. Given a parameter w_j , the update rules at iteration t are:

$$m_j^{(t)} = \beta_1 \cdot m_j^{(t-1)} + (1-\beta_1) \cdot g_j^{(t)}, \\ v_j^{(t)} = \beta_2 \cdot v_j^{(t-1)} + (1-\beta_2) \cdot (g_j^{(t)})^2, \\ \hat{m}_j^{(t)} = \frac{m_j^{(t)}}{1-\beta_1^t}, \hat{v}_j^{(t)} = \frac{v_j^{(t)}}{1-\beta_2^t}, \\ w_j^{(t+1)} = w_j^{(t)} - \frac{\eta}{\sqrt{\hat{v}_j^{(t)} + \epsilon}} \cdot \hat{m}_j^{(t)}$$

where $g_j^{(t)}$ is the gradient of the loss with respect to w_j , and $\beta_1, \beta_2 \in [0,1]$ are decay rates for the moment estimates. The bias corrected estimates $\hat{m}, \hat{v}$ ensure stability in the early stages of training.

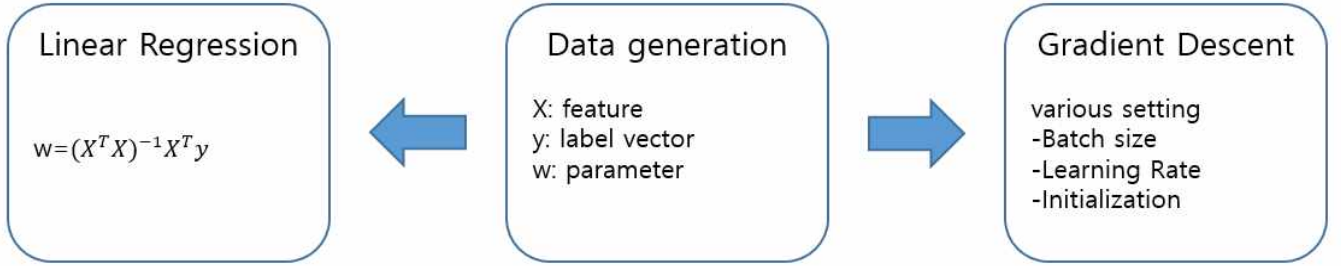


Figure 2. Overview of the experimental pipeline. The generated data is used to compute the solution by the linear regression formula, and the gradient descent method under various settings.

3. Problem1

Generate N linear regression samples, and its label vector using pre-determined parameters w . For analysis, use D -dimensional vectors with large number of samples ($N \geq 1000$, $D \geq 4$).

1) Find the solution using the linear regression method, and measure its estimation error.

-3.2.1 Linear Regression

2) Find the solution using gradient descent method and analyze its performance by various adaptive learning rate schemes, batch size, and initialization methods, etc.

-3.2.2 Gradient Descent

3.1 Method

Figure 2 illustrates our experimental setup. We first generate the required data samples, then estimate the parameter w using the solution of linear regression. Subsequently, we apply the gradient descent method under various settings to estimate the same parameter. The results from both methods are compared based on their estimation accuracy.

1) Data Generation.

```

N = 1000          # number of samples
D = 4             # number of features

X = np.random.rand(N, D)    # samples
w_true = np.random.rand(D)  # true weights
noise = 0.1 * np.random.randn(N)  # noise
y = X @ w_true + noise      # label
  
```

To evaluate the performance of regression models under controlled conditions, we generated a synthetic dataset adhering to a linear model with additive Gaussian noise. The input matrix $X \in \mathbb{R}^{N \times D}$ was composed of $N=1000$ samples, each with $D=4$ features, drawn independently from a uniform distribution over the interval $[0,1]$. The true weight vectors $w_{true} \in \mathbb{R}^D$ are randomly initialized.

To simulate real-world measurement errors, zero-mean Gaussian noise was added to the target values. Specifically, the noise vector $\epsilon \in \mathbb{R}^N$ was sampled from a normal distribution with standard deviation $\sigma=0.1$.

The corresponding label vector $y \in \mathbb{R}^N$ is computed using the linear model:

$$y = Xw_{true} + \epsilon$$

2) Linear Regression Solution.

```

w_pred = np.linalg.inv(X.T @ X) @ X.T @ y
y_pred = X @ w_pred
estimation_error = np.mean((y - y_pred) ** 2)
weight_error = np.linalg.norm(w_pred - w_true)
  
```

To evaluate the predictive performance of the linear regression model, we first computed the optimal weight vector using the closed-form solution of Ordinary Least Squares.

$$w_{pred} = \text{np.linalg.inv}(X.T @ X) @ X.T @ y$$

Using the estimated weights, the predicted labels were obtained via matrix multiplication:

$$y_{pred} = X @ w_{pred}$$

To quantify the model's predictive accuracy, we computed the Mean Squared Error (MSE) between the predicted outputs and the true labels:

$$estimation_error = \text{np.mean}((y - y_{pred}) ** 2)$$

The MSE provides a scalar measure of the average squared discrepancy between predicted and actual values.

To assess the accuracy of parameter recovery, we computed the estimation error as the Euclidean distance between the estimated weight vector and the ground-truth weights:

$$weight_error = \text{np.linalg.norm}(w_{pred} - w_{true})$$

This scalar value reflects the magnitude of the discrepancy between the learned model parameters and the underlying true parameters used to generate the data.

3) Gradient Descent

```

if init_type == 'random':
    w = np.random.randn(D)
elif init_type == 'zero':
    w = np.zeros(D)
elif init_type == 'sparse':
    w = np.random.randn(D)
    w[np.random.rand(D) < 0.8] = 0

for epoch in range(1, num_epochs + 1):
    if batch_type == 'full':
        X_batch = X
        y_batch = y
    elif batch_type == 'mini-batch':
        indices = np.random.choice(N, batch_size,
replace=False)
        X_batch = X[indices]
        y_batch = y[indices]
    elif batch_type == 'SGD':
        idx = np.random.randint(0, N)
        X_batch = X[idx:idx+1]
        y_batch = y[idx:idx+1]

    y_pred = X_batch @ w
    error = y_batch - y_pred
    gradient = (2 / X_batch.shape[0]) * X_batch.T @ error

    if optimizer_type == 'GD':
        w -= learning_rate * gradient
    elif optimizer_type == 'AdaGrad':
        grad_squared += gradient ** 2
        w -= learning_rate * gradient / (np.sqrt(grad_squared) + epsilon)
    elif optimizer_type == 'RMSProp':
        running_avg = 0.9 * running_avg + 0.1 * (gradient ** 2)
        w -= learning_rate * gradient / (np.sqrt(running_avg) + epsilon)
    elif optimizer_type == 'Adam':
        m = beta1 * m + (1 - beta1) * gradient
        v = beta2 * v + (1 - beta2) * (gradient ** 2)
        m_hat = m / (1 - beta1 ** epoch)
        v_hat = v / (1 - beta2 ** epoch)
        w -= learning_rate * m_hat / (np.sqrt(v_hat) + epsilon)

```

we implement a modular training loop that allows configuration of the batch type, initialization, and optimizer.

Batch Selection Strategy. Depending on the specified batch_type, we sample the training data accordingly:

Full batch: All samples are used at each step.

Mini-batch: A random subset of samples is selected. (We used a mini-batch size of 32)

SGD (Stochastic Gradient Descent): A single sample is used per update.

Gradient Computation. For the selected batch, we compute the mean squared error gradient:

$$(2 / X_batch.shape[0]) * X_batch.T @ error$$

Optimizer Update Rules. Based on the optimization strategies discussed in *Related work*, we implemented four methods.

Gradient Descent (GD)

Updates the model parameters using the full gradient with a fixed learning rate.

AdaGrad

Adapts the learning rate for each parameter individually based on the accumulated squared gradients, allowing larger updates for infrequent features.

RMSProp

Maintains an exponentially decaying average of squared gradients to adjust learning rates dynamically, preventing the learning rate from shrinking too quickly.

Adam

Combines momentum (first moment) and adaptive learning rates (second moment) with bias correction, providing fast and stable convergence across a wide range of tasks.

3.2 Performance

3.2.1 Linear Regression

In addition to estimating label estimation error, we also computed the weight residuals and correlation between the predicted and true labels.

Metric	Value
True Weights	0.29, 0.57, 0.14, 0.35
Estimated Weights	0.31, 0.56, 0.11, 0.36
Weight error(L2 norm)	0.024
Estimation error	0.010

Table 1. performance of Linear Regression

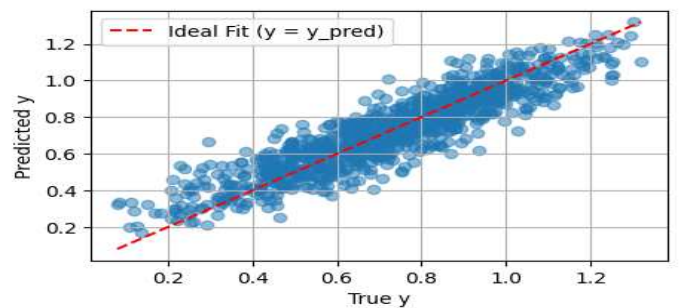


Figure 3. Correlation between true and predicted labels

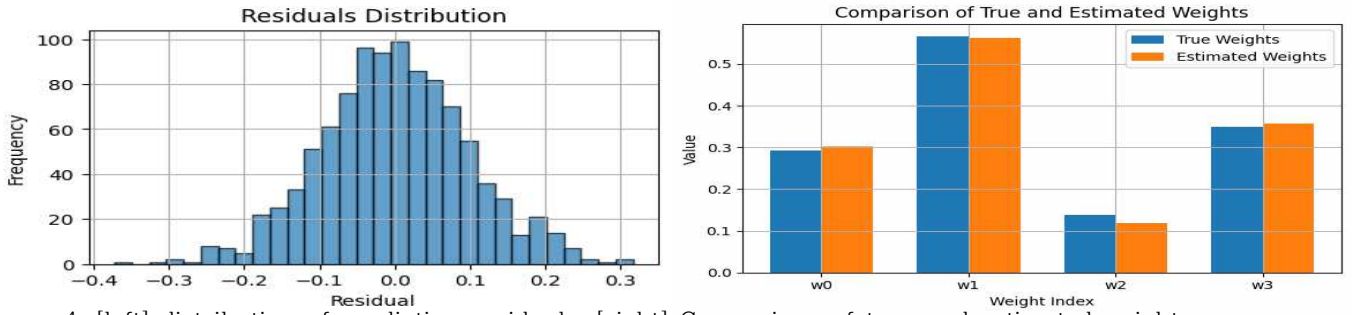


Figure 4. [left] distribution of prediction residuals, [right] Comparison of true and estimated weights

3.2.2 Gradient Descent(E: estimation error, W: weight error)

Optimizer	Batch	Init	E	W	Optimizer	Batch	Init	E	W
GD	SGD	random	0.003777	0.292846	RMSProp	SGD	random	0.012550	0.066291
		sparse	0.014131	0.342622			sparse	0.008390	0.076043
		zero	0.000006	0.063122			zero	0.005268	0.063571
	Full	random	0.018023	0.323304		Full	random	0.010386	0.026968
		sparse	0.020539	0.354951			sparse	0.010386	0.024255
		zero	0.010564	0.054999			zero	0.010386	0.026968
	mini	random	0.016387	0.327782		mini	random	0.011434	0.028385
		sparse	0.016905	0.357052			sparse	0.011434	0.028385
		zero	0.012469	0.056427			zero	0.011434	0.028385
AdaGrad	SGD	random	2.070222	1.923130	Adam	SGD	random	0.006793	0.256750
		sparse	0.000010	1.765343			sparse	0.019549	0.066129
		zero	0.005431	0.286664			zero	0.003076	0.047305
	Full	random	1.147051	1.803198		Full	random	0.010787	0.096063
		sparse	0.219942	1.551531			sparse	0.010278	0.023618
		zero	0.016561	0.258904			zero	0.010278	0.023618
	mini	random	1.276278	1.809745		mini	random	0.011594	0.103269
		sparse	0.210689	1.575516			sparse	0.011445	0.022336
		zero	0.019882	0.260425			zero	0.011434	0.023010

Table 2. learning rate=0.01

Optimizer	Batch	Init	E	W	Optimizer	Batch	Init	E	W
GD	SGD	random	0.010076	0.095096	RMSProp	SGD	random	0.000388	0.279109
		sparse	0.004776	0.113207			sparse	0.007111	0.371616
		zero	0.003071	0.096848			zero	0.000200	0.196500
	Full	random	0.010278	0.023618		Full	random	0.021075	0.099312
		sparse	0.010278	0.023618			sparse	0.021075	0.099312
		zero	0.010278	0.023618			zero	0.021075	0.106079
	mini	random	0.011680	0.023311		mini	random	0.031219	0.116162
		sparse	0.011680	0.023310			sparse	0.031219	0.116162
		zero	0.011680	0.023310			zero	0.031219	0.116162
AdaGrad	SGD	random	0.000559	0.501811	Adam	SGD	random	0.008111	0.287254
		sparse	0.015135	0.204853			sparse	0.000779	0.276392
		zero	0.003532	0.038763			zero	0.006187	0.411862
	Full	random	0.012392	0.177239		Full	random	0.010278	0.023618
		sparse	0.010278	0.023757			sparse	0.010278	0.023618
		zero	0.010278	0.023618			zero	0.010278	0.023618
	mini	random	0.012845	0.202843		mini	random	0.013083	0.026531
		sparse	0.011656	0.023797			sparse	0.012611	0.045427
		zero	0.011648	0.023572			zero	0.013156	0.030971

Table 3. learning rate=0.1

learning rate 0.01(left) vs 0.1(right)

Figure 5. Average MSE by optimizer

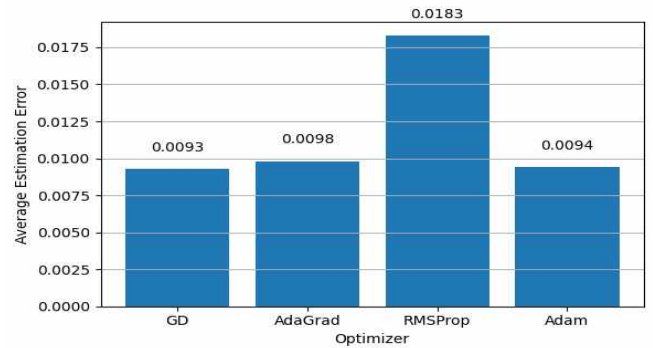
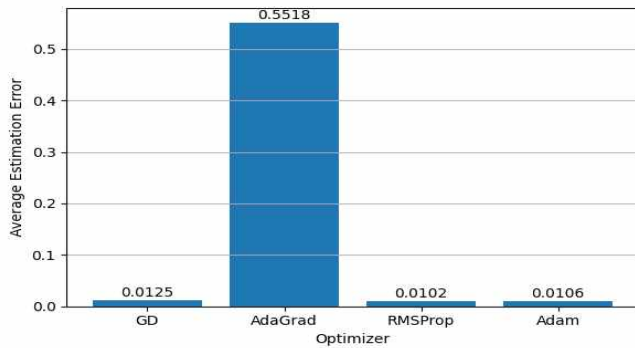


Figure 6. MSE over Epochs, Batch=SGD, Init=sparse

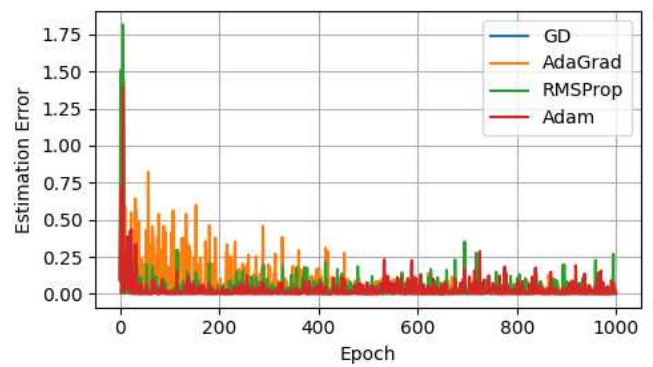
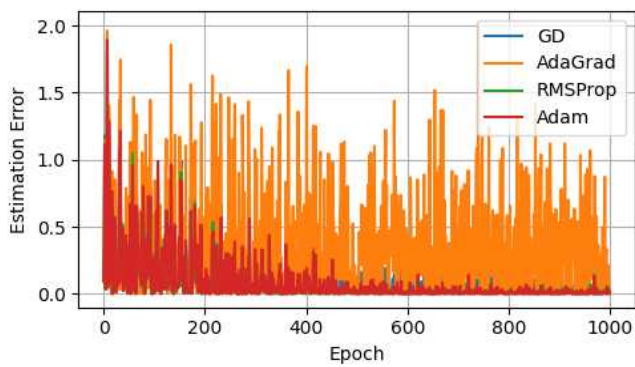


Figure 7. MSE over Epochs, Batch=mini, Init=random

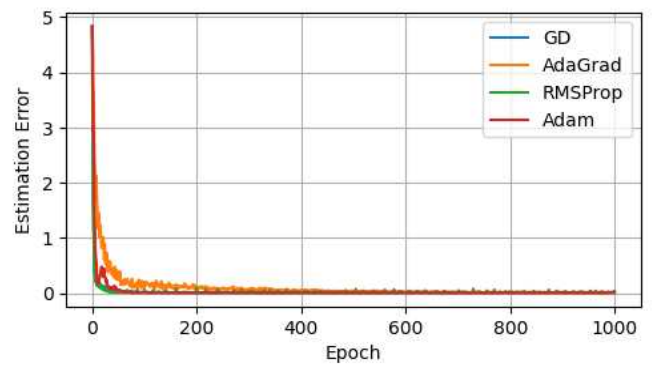
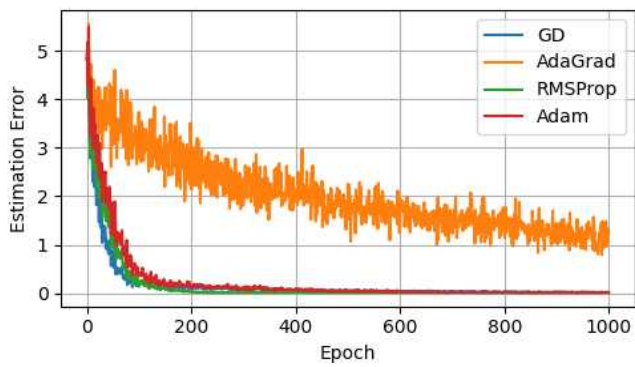
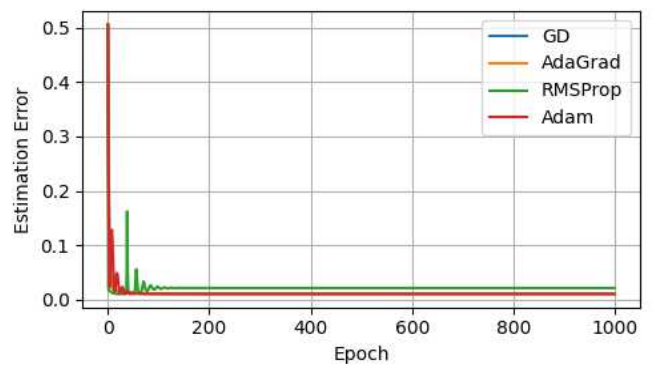
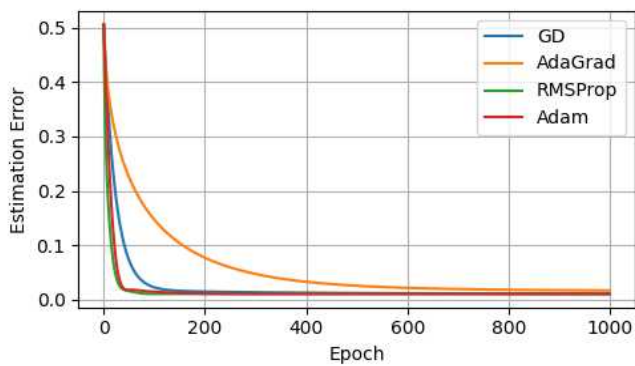


Figure 8. MSE over Epochs, Batch=full, Init=zero



learning rate 0.01(left) vs 0.1(right)

Figure 9. Average Weight error by optimizer

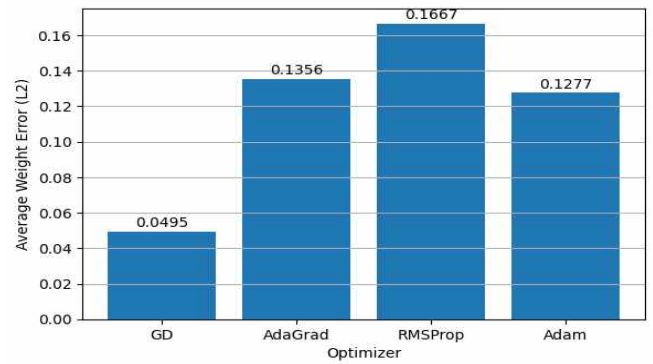
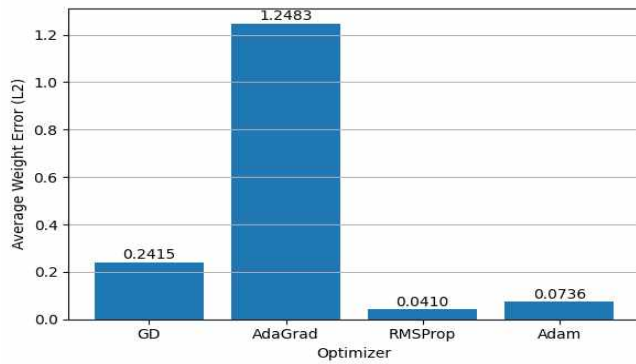


Figure 10. Weight Error over Epochs, Batch=SGD, Init=sparse

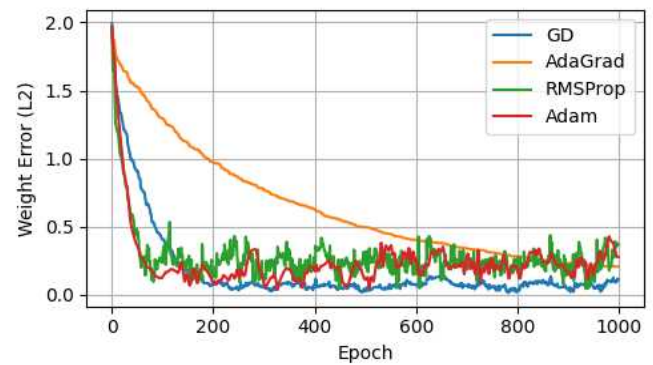
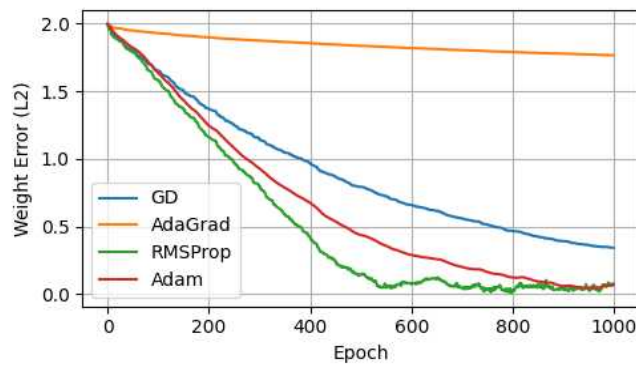


Figure 11. Weight Error over Epochs, Batch=mini, Init=random

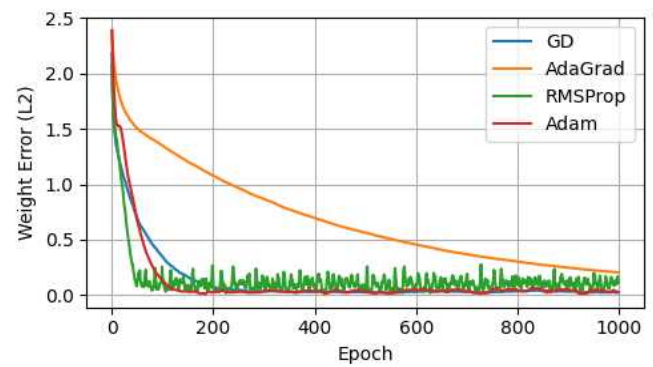
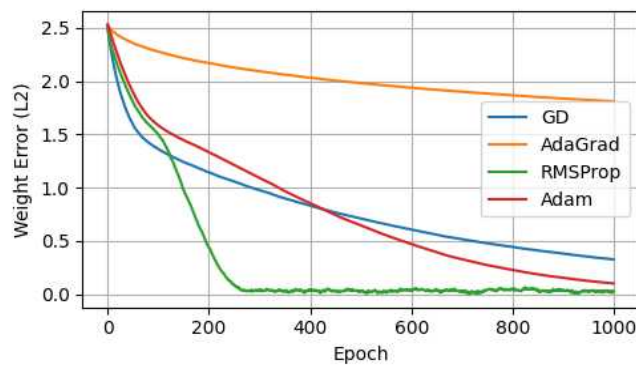
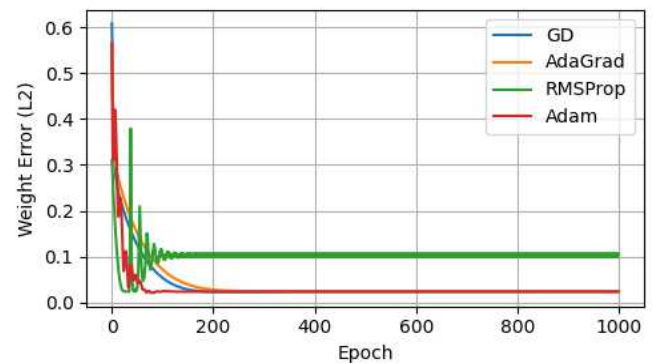
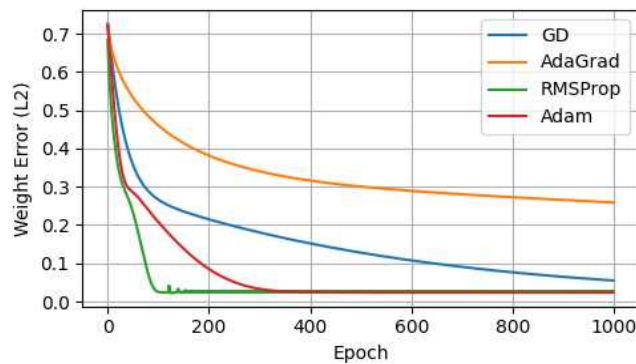


Figure 12. Weight over Epochs, Batch=full, Init=zero



3.3 Performance Analysis

3.3.1 Linear Regression

To establish a baseline for comparison, we first evaluated the closed-form solution of ordinary least squares (OLS) regression on a synthetic dataset. The estimated weight vector showed strong agreement with the ground truth, as summarized in Table 1. Specifically, the label estimation error was 0.010, and the weight estimation error, measured as the L2 norm between the true and predicted weight vectors, was 0.024. Figure 3 illustrates the high correlation between the true and predicted labels, confirming the model's ability to capture the underlying data-generating process. In addition, residuals were symmetrically distributed around zero (Figure 4, left), and a side-by-side comparison of true and predicted weights for each dimension (Figure 4, right) further demonstrated the consistency of parameter recovery.

3.3.2 Gradient Descent-based Estimation

To explore the behavior of iterative learning algorithms, we implemented and evaluated four optimizers—Gradient Descent (GD), AdaGrad, RMSProp, and Adam across different configurations of batch types (SGD, mini-batch, full-batch), weight initialization strategies (random, zero, sparse), and learning rates (0.01 and 0.1). For each setup, we tracked two key metrics: label estimation error (E) and weight error (W), which quantify predictive performance and parameter recovery accuracy, respectively.

Learning Rate = 0.01

As shown in Table 2 and Figure 5 (left), the Adam and RMSProp optimizers consistently yielded the lowest estimation and weight errors across most settings. In contrast, AdaGrad performed poorly with sparse weight initialization, resulting in an average estimation error exceeding 0.05 and a weight error above 1.27—the highest among all optimizers. SGD showed highly variable behavior, particularly with sparse initialization, often failing to converge stably (Figures 6 and 7). Gradient Descent, while slower, exhibited stable and reliable convergence, especially in the full-batch setting (Figure 8, left).

Learning Rate = 0.1 (Table 3)

Increasing the learning rate to 0.1 led to faster convergence but introduced potential instability in some configurations. Table 3 and Figure 5 (right) show that GD maintained both low label estimation error (0.0093) and low weight error (0.0495). Adam continued to perform robustly, particularly under mini-batch and full-batch settings, while AdaGrad showed improved performance compared to the 0.01 setting, though still lagging behind Adam and RMSProp. Notably, RMSProp showed slightly elevated

average errors (estimation error ≈ 0.0183) at this learning rate, suggesting sensitivity to hyperparameter tuning.

3.3.3 Training Dynamics

We also analyzed the learning trajectories over epochs, as visualized in Figures 6 through 12. In terms of label estimation error (Figures 6–8), Adam and RMSProp converged quickly and stably in most settings. GD exhibited smooth but relatively slower convergence, while AdaGrad showed considerable fluctuation under sparse initialization. Weight error trends (Figures 10–12) revealed similar patterns: Adam was the most consistent optimizer, achieving the lowest final weight error across various configurations. In contrast, AdaGrad often plateaued early, particularly when applied to sparse input features.

3.3.4 Summary

Overall, Adam emerged as the most robust and efficient optimizer across all tested configurations, offering fast convergence and low final errors regardless of initialization or batch size. RMSProp showed comparable performance with slightly higher variance.

4. Problem2

Repeat the problem #1 using two parameter vectors w_1, w_2 , i.e., generated the label data using either w_1 or w_2 as the following figure. Here, the labeled data y_i are constructed using the parameter vector w_i with the probability p_i , and $p_1 + p_2 = 1$. Let $p_1 = 90\%$.

Ideal scenario: labeled data information is known

1) Find the linear regression solution using y_1 labeled data only, and measure its estimation error. (It will be the performance upper bound.)

-4.2.1 linear regression using y_1 data only

Practical scenario: labeled data information is unknown.

2) Find the solution using all data samples, and measure its estimation error.

-4.2.2 Linear regression using all data samples

3) Use the gradient descent method and analyze its performance under various conditions.

-4.2.3 Use the gradient descent method

4) Analyze the performance degradation by outlier samples (y_2) from 1)–3). Check the performance by controlling the number of outlier samples, i.e., by adjusting p_1 .

-4.2.4 Degradation by outlier samples

5) Improve the gradient descent method by considering the outlier samples as much as possible.

-4.2.5 Improving the gradient descent method

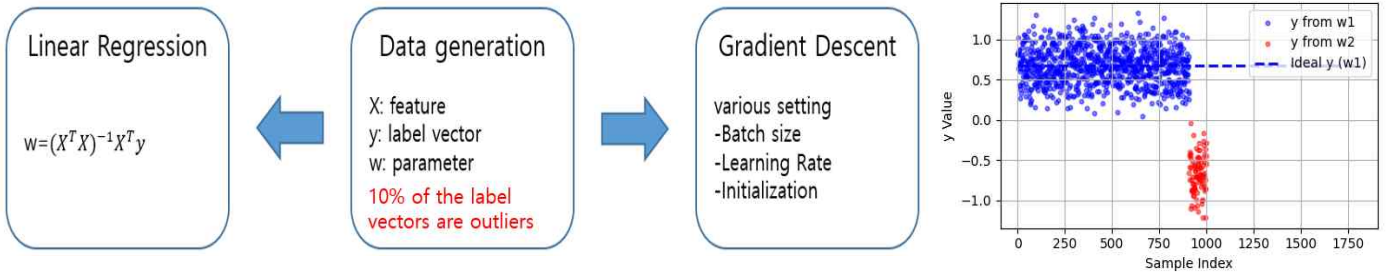


Figure 13. Overview of the Problem 2. "In contrast to Problem 1, the label vector generation in this task involves the outliers.

4.1 Method

For Problems 2.3 and 2.4, we reused the linear regression and gradient descent implementations from Problem 1 without modification.

In this work, we generate data and apply both analytical linear regression and various gradient descent methods to solve the regression task. We then investigate how outliers affect the model's performance and, based on our findings, design a custom approach to enhance the model's robustness and predictive accuracy.

1) Data Generation.

```
N, D = 1000, 4
p = 0.9
X = np.random.rand(N, D)
w1 = np.random.rand(D)
w2 = -w1
noise = 0.1 * np.random.randn(N)

y = np.zeros(N)
z = np.zeros(N, dtype=int)

for i in range(N):
    if np.random.rand() < p:
        y[i] = X[i] @ w1 + noise[i]
        z[i] = 1
    else:
        y[i] = X[i] @ w2 + noise[i]
        z[i] = 2
```

We generated a synthetic dataset using a mixture of two opposing linear models. Each of the $N=1000$ samples with $D=4$ features was assigned to one of two models with probability $p=0.9$ or $1-p$, using weight vectors w_1 and $w_2 = -w_1$, respectively. Gaussian noise was added to simulate observational error. The resulting dataset reflects heterogeneity in the data-generating process, suitable for testing model robustness under mixed distributions.

2) Linear Regression Solution_Ideal scenario

To evaluate the model's ability to recover the underlying parameters for a specific subpopulation,

we performed regression solely on the samples generated by the first linear model (i.e., those with $z=1$).

```
X_y1 = X[z == 1]
y_y1 = y[z == 1]
w_y1 = np.linalg.pinv(X_y1) @ y_y1

y_pred = X @ w_y1
y_pred_y1 = y_pred[z == 1]

estimation_error = np.mean((y_y1 - y_pred_y1)** 2)
weight_error = np.linalg.norm(w_y1 - w1)
```

A weight vector w_{y1} was estimated using the Moore-Penrose pseudoinverse. The remaining procedure follows the same methodology as described in Section 3.1 (Problem 1).

3) Linear Regression Solution_Practical scenario

```
w_all = np.linalg.pinv(X) @ y
y_pred = X @ w_all
y_y1 = y[z == 1]
y_pred_y1 = y_pred[z == 1]

estimation_error = np.mean((y_y1 - y_pred_y1) ** 2)
weight_error = np.linalg.norm(w_all - w1)
```

The goal of this procedure is to assess how well a regression model trained on the entire dataset, without regard to its underlying heterogeneity, performs when evaluated specifically on samples generated from only one of the underlying processes (i.e., group $z=1$).

4) gradient descent method

we modified part of the original code to compute the error exclusively for the first generative model.

```
<before>
error_history.append(np.mean((y_batch-y_pred) ** 2))

<after>
mse_history.append(np.mean((y[z==1]-y_pred_all[z==1])
** 2))
```

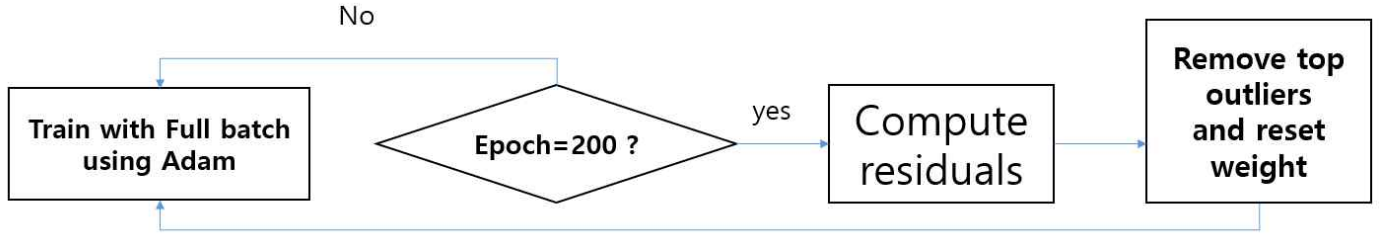


Figure 14. Overview of our method, improving the gradient descent method

5) Improve the gradient descent method

We reformulated the linear regression task as an outlier removal problem to improve model robustness. Empirical observations during training revealed that most gradient-based methods, particularly under the Adam optimizer with full-batch updates, converged after approximately 200 epochs. Building on this insight, we implemented an iterative refinement strategy wherein, after every 200 epochs, the samples whose target values deviated the most from the current regression hyperplane were treated as outliers and removed. Specifically, a fixed proportion of the samples with the largest residuals were excluded, and the model was retrained from scratch using the remaining data.

This procedure was repeated iteratively, and was based on the Adam (full-batch) configuration, which consistently demonstrated the best performance among all tested gradient descent variants. The objective was to reduce the influence of extreme observations and refine the model to better capture the underlying linear relationship present in the majority of the data.

```

if epoch % reset_interval == 0 and epoch != num_epochs:
    residuals = np.abs(X_current @ w - y_current)
    cutoff = np.percentile(residuals, 100 * (1 - outlier_ratio))
    inlier_mask = residuals <= cutoff

    X_current = X_current[inlier_mask]
    y_current = y_current[inlier_mask]
    z_current = z_current[inlier_mask]
  
```

At fixed intervals defined by *reset_interval*, we computed the absolute residuals between the predicted and true labels:

$$residuals = np.abs(X_{current} @ w - y_{current})$$

Next, a threshold value (cutoff) was determined by computing the $(1 - outlier_ratio) \times 100$ percentile of the residual distribution:

$$cutoff = np.percentile(residuals, 100 * (1 - outlier_ratio))$$

This percentile-based threshold serves to identify and

remove the top proportion of samples exhibiting the largest residuals. Specifically, only the samples with residuals below or equal to the cutoff were retained for subsequent training:

$$\begin{aligned}
 inlier_mask &= residuals \leq cutoff \\
 X_{current} &= X_{current}[inlier_mask] \\
 y_{current} &= y_{current}[inlier_mask] \\
 z_{current} &= z_{current}[inlier_mask]
 \end{aligned}$$

This mechanism was applied after every *reset_interval* epochs, excluding the final epoch. By iteratively removing high-error samples and retraining on the inliers, the model is encouraged to fit the dominant underlying pattern in the data, thereby mitigating the influence of extreme outliers.

4.2 Performance

4.2.1 Linear regression using y_1 data only

All numerical results are identical to those in '3.2.1 Linear Regression', which serves as an upper bound for Problem 2.

Metric	Value
True Weights	0.29, 0.57, 0.14, 0.35
Estimated Weights	0.31, 0.56, 0.11, 0.36
Weight error(L2 norm)	0.024
Estimation error	0.010

Table 4. Linear Regression using y_1 only

The linear regression model trained with y_1 alone demonstrated high accuracy, with an L2 norm weight error of 0.024 and an estimation error of 0.010, suggesting reliable parameter estimation.

4.2.2 Linear regression using all data samples

Metric	Value
True Weights	0.29, 0.57, 0.14, 0.35
Estimated Weights	0.24, 0.51, 0.07, 0.27
Weight error(L2 norm)	0.131
Estimation error	0.027

Table 5. Linear Regression using all data samples

The linear regression model trained on all data samples showed a higher weight error (0.131) and estimation error (0.027) compared to the model using only y_1 , suggesting that including all samples introduced higher error in parameter estimation.

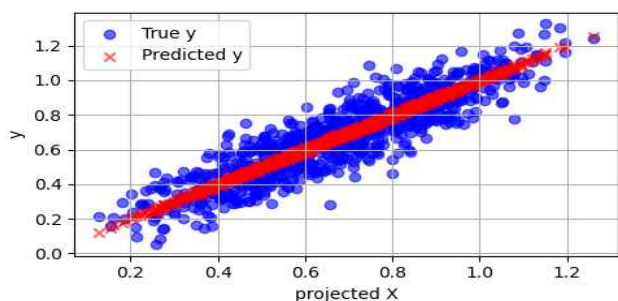


Figure 15. True and Predicted y, using y1 only

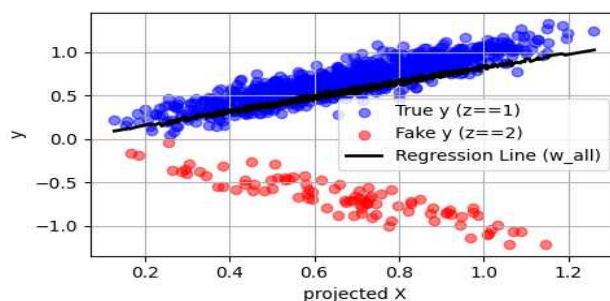


Figure 16. True and Predicted y, using all data

4.2.3 Use the gradient descent method (E: estimation error, W: weight error)

Optimizer	Batch	Init	E	W	Optimizer	Batch	Init	E	W
GD	SGD	random	0.056094	0.309221	RMSProp	SGD	random	0.034741	0.208574
		sparse	0.084162	0.363039			sparse	0.056872	0.243299
		zero	0.068330	0.250949			zero	0.043116	0.202950
	Full	random	0.034301	0.333413		Full	random	0.030244	0.141166
		sparse	0.036525	0.360079			sparse	0.024803	0.121316
		zero	0.027757	0.140556			zero	0.030244	0.141166
	mini	random	0.036292	0.334624		mini	random	0.029869	0.145631
		sparse	0.038206	0.356872			sparse	0.029869	0.145631
		zero	0.030535	0.150817			zero	0.029870	0.145631
AdaGrad	SGD	random	1.623103	1.395141	Adam	SGD	random	0.087271	0.445996
		sparse	0.288897	1.805406			sparse	0.105377	0.343842
		zero	0.071259	0.364454			zero	0.117065	0.322247
	Full	random	1.107759	1.798833		Full	random	0.027756	0.164101
		sparse	0.248064	1.653059			sparse	0.027416	0.131234
		zero	0.033642	0.265535			zero	0.027416	0.131236
	mini	random	1.129720	1.804901		mini	random	0.033065	0.180166
		sparse	0.256027	1.674494			sparse	0.048191	0.192906
		zero	0.036660	0.281441			zero	0.049035	0.195971

Table 6. learning rate=0.01

Optimizer	Batch	Init	E	W	Optimizer	Batch	Init	E	W
GD	SGD	random	0.030857	0.458112	RMSProp	SGD	random	0.026022	0.420945
		sparse	0.291018	0.837345			sparse	0.246802	0.740603
		zero	0.100870	0.586586			zero	0.077545	0.466659
	Full	random	0.027416	0.131236		Full	random	0.065379	0.230811
		sparse	0.027416	0.131236			sparse	0.010970	0.034232
		zero	0.027416	0.131236			zero	0.065379	0.230811
	mini	random	0.027619	0.138056		mini	random	0.012840	0.153736
		sparse	0.027619	0.138056			sparse	0.012840	0.153736
		zero	0.027619	0.138056			zero	0.012840	0.153736
AdaGrad	SGD	random	0.068879	0.678092	Adam	SGD	random	0.078813	0.715660
		sparse	0.063494	0.466583			sparse	0.264696	0.868063
		zero	0.043840	0.193060			zero	0.475249	0.886525
	Full	random	0.029650	0.238513		Full	random	0.027416	0.131236
		sparse	0.027402	0.131100			sparse	0.027416	0.131236
		zero	0.027416	0.131236			zero	0.027416	0.131236
	mini	random	0.033120	0.268928		mini	random	0.016753	0.141961
		sparse	0.032715	0.149962			sparse	0.015352	0.168200
		zero	0.032956	0.151957			zero	0.015464	0.168367

Table 7. learning rate=0.1

Estimation error. learning rate 0.01(left) vs 0.1(right)

Figure 17. Batch: Full Init: Sparse

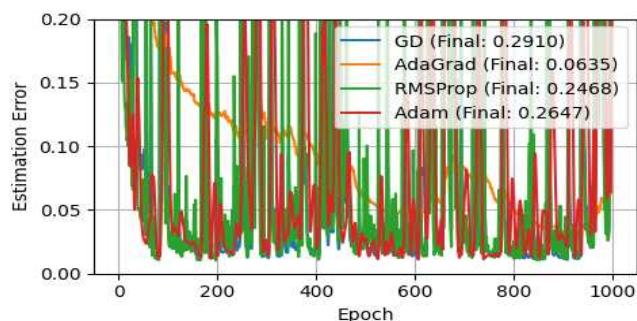
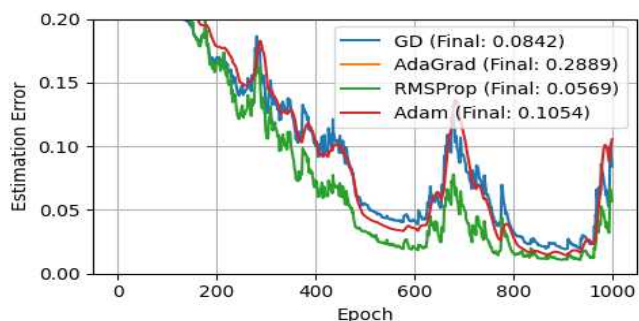


Figure 18. Batch: SGD Init: Zero

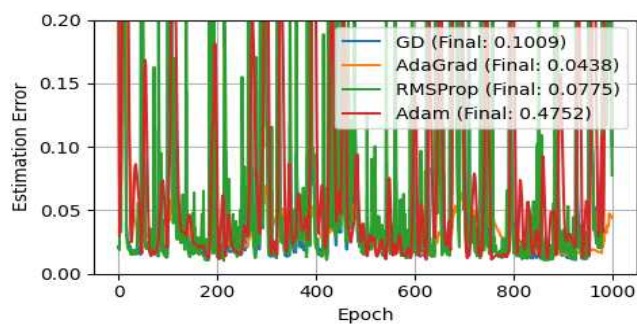
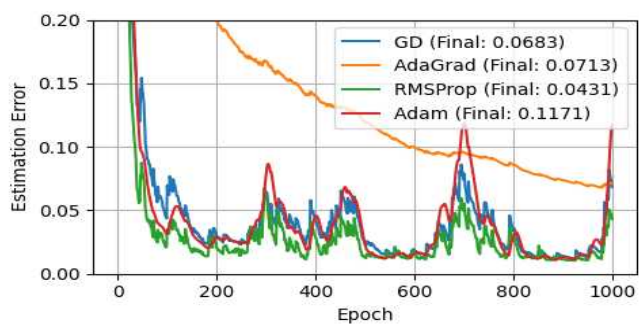


Figure 19. Batch: Mini Init: Random

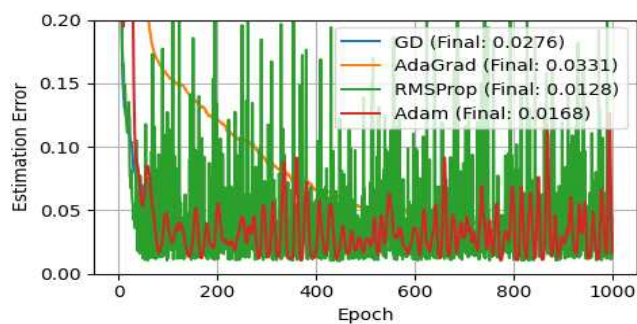
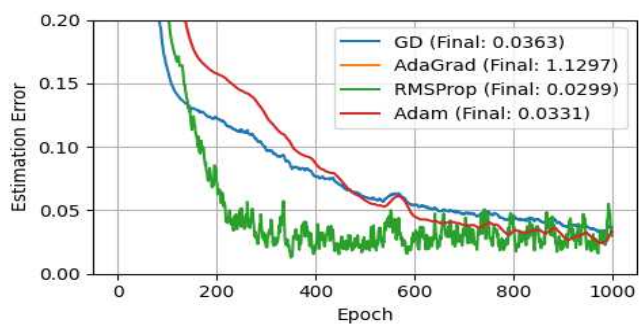
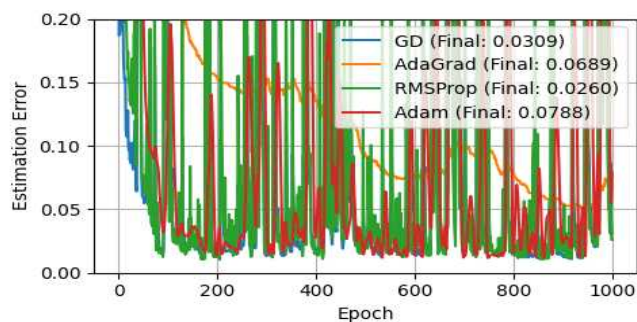
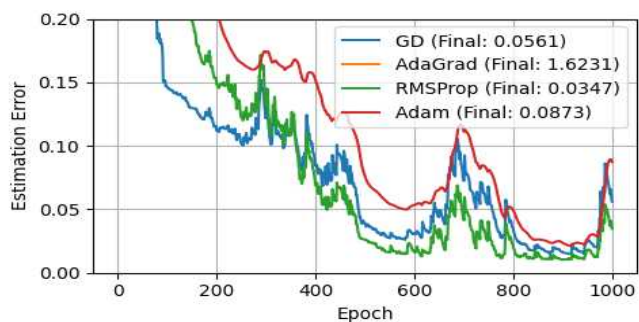


Figure 20. Batch: SGD Init: Random



Weight Error. Learning rate 0.01(left) vs 0.1(right)

Figure 21. Batch: Full Init: Sparse

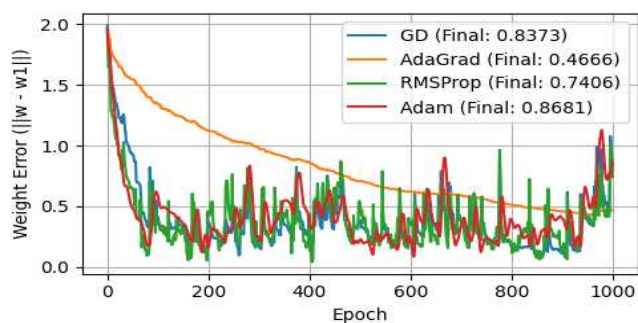
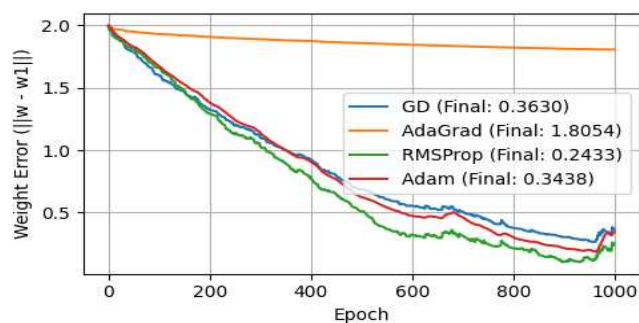


Figure 22. Batch: SGD Init: Zero

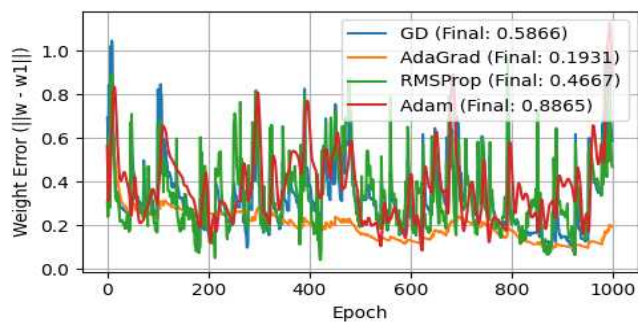
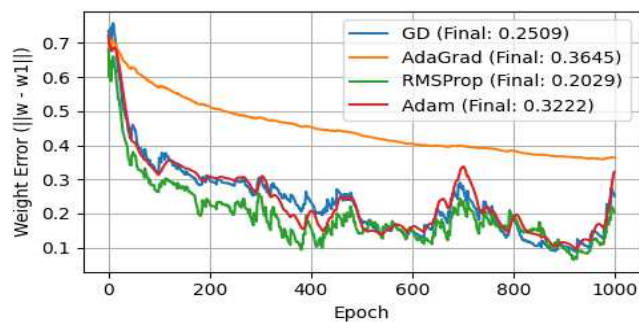


Figure 23. Batch: Mini Init: Random

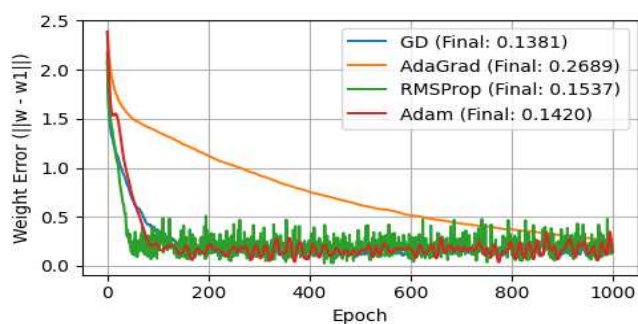
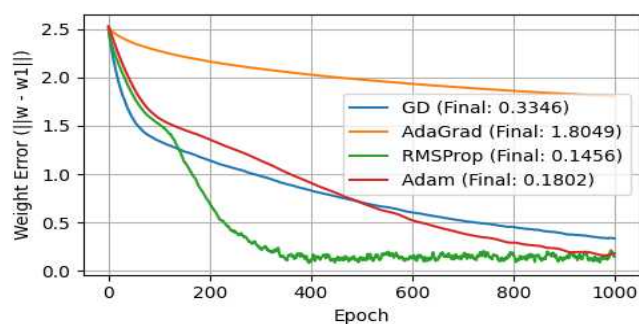
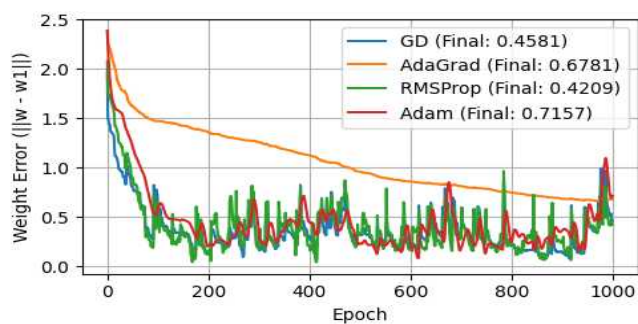
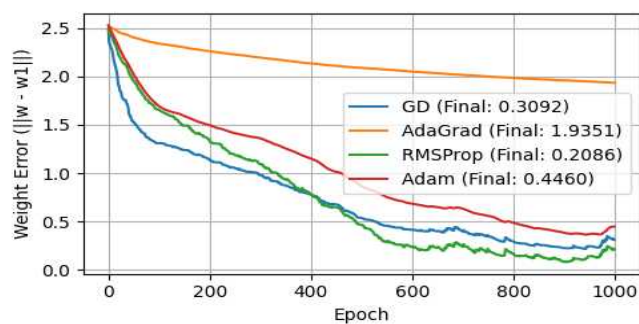


Figure 24. Batch: SGD Init: Random



4.2.4 Degradation by outlier samples

To investigate the robustness of different optimization strategies, we gradually increased the probability of outlier contamination and examined its impact on the estimation error of both the analytical linear regression solution and gradient descent-based approaches.

Degradation in Linear Regression

Estimation Error	
probability	Value
0%	0.010
10%	0.027
30%	0.193
50%	0.508

Table 8. The estimation error increases as the outlier probability p rises.

Weight error	
probability	Value
0%	0.024
10%	0.131
30%	0.428
50%	0.717

Table 9. The weight error increases as the outlier probability p rises.

As shown in Figure 25, both the estimation error and weight error exhibit a clear upward trend as p increases, indicating progressive degradation of model performance.

Quantitatively, Table 8 presents the growth in

estimation error. When $p=0\%$, the model achieves a low error of 0.010. However, as the outlier probability increases to 10%, 30%, and 50%, the error rises to 0.027, 0.193, and 0.508.

Similarly, Table 9 shows the behavior of the weight estimation error, which increases from 0.024 at $p=0\%$ to 0.717 at $p=50\%$. Notably, the weight error increases more gradually than the estimation error in the early stages, but both metrics eventually exhibit sharp growth as p exceeds 30%.

These results demonstrate that standard linear regression, when trained on data that includes an increasing proportion of samples from conflicting distributions, becomes increasingly inaccurate in both prediction and parameter recovery.

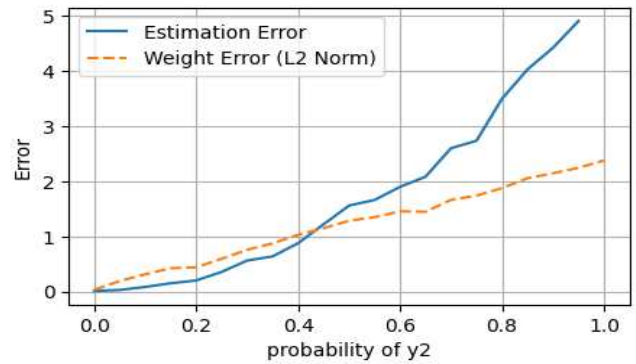


Figure 25. Both the estimation error and weight error increase as the outlier probability p rises.

Degradation in Gradient Descent Method

Each value represents the degree of performance degradation relative to the clean baseline ($p = 0\%$)

Optimizer	Batch	Init	p=10	p=20	p=30	Optimizer	p=10	p=20	p=30
GD	SGD	random	0.052317	0.146806	0.243516	RMSProp	0.022191	0.104072	0.209230
		sparse	0.070031	0.159883	0.244164		0.048482	0.127495	0.224411
		zero	0.068324	0.158091	0.263168		0.037848	0.110561	0.217556
	Full	random	0.016278	0.081810	0.185458		0.019858	0.086230	0.191687
		sparse	0.015986	0.079640	0.183262		0.014417	0.074477	0.173987
		zero	0.017193	0.080825	0.183515		0.019858	0.086230	0.173987
	mini	random	0.019905	0.083156	0.184721		0.018435	0.083195	0.206414
		sparse	0.021301	0.082774	0.184175		0.018435	0.083194	0.206414
		zero	0.018066	0.080019	0.181110		0.018436	0.083194	0.206414
AdaGrad	SGD	random	0.447119	0.443690	0.453772	Adam	0.080478	0.152711	0.252094
		sparse	0.288887	0.298879	0.320196		0.085828	0.168137	0.251035
		zero	0.065828	0.140911	0.264019		0.113989	0.153549	0.225400
	Full	random	0.039292	0.050733	0.066600		0.016969	0.081451	0.185009
		sparse	0.028122	0.103496	0.188821		0.017138	0.080351	0.182826
		zero	0.017081	0.078294	0.178569		0.017138	0.080353	0.182837
	mini	random	0.146558	0.161353	0.173662		0.021471	0.081870	0.184110
		sparse	0.045338	0.115092	0.190967		0.036746	0.106927	0.211689
		zero	0.016778	0.079866	0.177387		0.037601	0.111927	0.220812

Table 8. Error growth rate compared to the Clean Baseline, learning rate=0.01

Optimizer	Batch	Init	p=10	p=20	p=30	Optimizer	p=10	p=20	p=30
GD	SGD	random	0.020781	0.233337	0.234822	RMSProp	0.025634	0.191582	0.197000
		sparse	0.286242	0.310404	0.327481		0.239691	0.264422	0.277459
		zero	0.097799	0.491424	0.689700		0.077345	0.431251	0.635953
	Full	random	0.017138	0.080353	0.182837		0.044304	0.139119	0.271367
		sparse	0.017138	0.080353	0.182837		0.010105	0.021589	0.094365
		zero	0.017138	0.080353	0.182837		0.044304	0.139119	0.271367
	mini	random	0.015939	0.079615	0.210351		0.018379	0.093473	0.251343
		sparse	0.015939	0.079615	0.210351		0.018379	0.093473	0.251343
		zero	0.015939	0.079615	0.210351		0.018379	0.093473	0.251343
AdaGrad	SGD	random	0.068320	0.148085	0.266941	Adam	0.070702	0.109623	0.326824
		sparse	0.048359	0.124771	0.234017		0.263917	0.230394	0.201659
		zero	0.040308	0.102663	0.202741		0.469062	0.750787	0.917165
	Full	random	0.017258	0.083366	0.188508		0.017138	0.080353	0.182837
		sparse	0.017124	0.080277	0.182648		0.017138	0.080353	0.182837
		zero	0.017138	0.080353	0.182837		0.017138	0.080353	0.182837
	mini	random	0.020275	0.084747	0.188039		0.003670	0.063071	0.276908
		sparse	0.021059	0.082896	0.189604		0.002741	0.037836	0.280390
		zero	0.021308	0.083701	0.191380		0.002308	0.035828	0.279543

Table 9. Error growth rate compared to the Clean Baseline, learning rate=0.1

Learning Rate = 0.01

As shown in Table 8, all optimizers exhibited a consistent increase in estimation error with rising p , though the rate of degradation varied significantly by optimizer and configuration. Among them, Adam and RMSProp demonstrated the highest robustness. For example, Adam with full batch and zero initialization showed only a moderate increase (0.182307) at $p=30\%$, whereas SGD and AdaGrad suffered from substantially higher error growth, particularly under sparse or zero initialization. Notably, AdaGrad with mini-batch and sparse initialization recorded a growth rate of 0.189604 at $p=30\%$, while SGD in similar settings reached as high as 0.243516.

These results indicate that adaptive optimizers such as Adam are better equipped to handle noisy data distributions, maintaining relatively low error increases even when a significant portion of the data deviates from the underlying model.

Learning Rate = 0.1

The effects of outlier contamination were further magnified under a higher learning rate, as presented in Table 9. While GD maintained relatively stable performance across initialization strategies, SGD and especially AdaGrad became more vulnerable to degradation. For instance, AdaGrad with SGD batch and zero initialization showed an error growth rate of 0.917165 at $p=30\%$, reflecting a more than nine-fold increase compared to the baseline.

In contrast, Adam remained the most stable optimizer, with significantly lower error increases under both full and mini-batch configurations. For

example, in the mini-batch with zero initialization case, Adam's growth rate was limited to 0.279543, despite a high learning rate and 30% outlier presence.

Conclusion

Across all experiments, we observed performance degradation as the probability p of data being generated from an opposing linear model increased.

In the case of standard linear regression, both the estimation error and weight error exhibited sharp growth with increasing p , as shown in Figure 25 and Tables 8 and 9. For example, the estimation error rose from 0.010 at $p=0\%$ to 0.508 at $p=50\%$, indicating a 50-fold degradation. This highlights the strong sensitivity of closed-form solutions to outliers and motivates the need for robust regression strategies.

A similar trend was observed in gradient-based methods. Through systematic analysis of various optimizers, batch types, and initialization strategies, we found that adaptive methods such as Adam and RMSProp consistently outperformed traditional GD, SGD, and AdaGrad. In particular, Adam demonstrated the most robust behavior across all scenarios, with relatively low error growth even under high contamination levels and large learning rates. In contrast, AdaGrad and SGD were highly sensitive to initialization conditions—especially under sparse settings—and suffered from accelerated degradation as the outlier ratio increased.

Overall, our results confirm that both analytical and iterative linear regression approaches are susceptible to performance degradation in the presence of outliers.

method	learning rate = 0.01			learning rate = 0.1			learning rate = 0.5		
	p=10	p=20	p=30	p=10	p=20	p=30	p=10	p=20	p=30
Linear Regression	0.027416	0.090631	0.193115	0.027416	0.090631	0.193115	0.027416	0.090631	0.193115
	0.024803	0.084863	0.184373	0.010970	0.042664	0.115440	0.014420	0.036086	0.021279
Top5	0.027416	0.090629	0.193104	0.012840	0.048984	0.192926	0.024467	0.043068	0.046844
GD	0.027756	0.090631	0.193115	0.015352	0.050447	0.193115	0.024502	0.055138	0.047483
method	0.027757	0.091389	0.193579	0.015464	0.076154	0.197388	0.026465	0.056298	0.051863
	0.029869	0.092238	0.194079	0.016753	0.090555	0.200884	0.027416	0.066752	0.100140
Ours	0.011940	0.011852	0.012298	0.010274	0.010180	0.010540	0.010273	0.010179	0.010539

Table 10. Estimation Error

method	learning rate = 0.01			learning rate = 0.1			learning rate = 0.5		
	p=10	p=20	p=30	p=10	p=20	p=30	p=10	p=20	p=30
Linear Regression	0.131236	0.287513	0.427894	0.131236	0.287513	0.427894	0.131236	0.287513	0.427894
	0.121316	0.277970	0.418365	0.034232	0.194157	0.333855	0.130289	0.240478	0.157871
Top5	0.131234	0.287504	0.427889	0.131100	0.281261	0.427850	0.130602	0.287513	0.268655
GD	0.131236	0.287513	0.427894	0.131236	0.283359	0.427894	0.131236	0.306695	0.270069
method	0.140556	0.297087	0.437443	0.138056	0.287205	0.443570	0.135101	0.306750	0.282679
	0.141166	0.299128	0.438501	0.141961	0.287513	0.443750	0.163562	0.307109	0.396479
Ours	0.148980	0.142240	0.148727	0.043356	0.033631	0.046142	0.043343	0.033644	0.046094

Table 11. Weight Error(L2 norm)

4.2.5 Improving the gradient descent method

In Section 4.1 Method_5) improve the gradient descent, we proposed an improvement to the standard gradient descent method by incorporating an outlier removal mechanism. Specifically, we adopted the Adam optimizer with full-batch updates and introduced an additional step that estimates and removes outliers at regular training intervals. This iterative process is illustrated in Figure 26.

To assess the effectiveness of our proposed method which incorporates periodic outlier removal into full-batch, Adam optimization, we compared its performance against baseline linear regression and gradient-based methods under varying outlier probabilities and learning rates. Tables 10 and 11 summarize the estimation and weight errors, respectively, for three learning rate settings (0.01, 0.1, 0.5) and outlier probabilities (10%, 20%, 30%).

Estimation Error (Table 10)

Across all learning rates and outlier probabilities, our method consistently achieved the lowest estimation error. For example, at a learning rate of 0.01 and $p=30\%$, our method produced an error of 0.012298, which is approximately 6% of the baseline linear regression error (0.193115). Similar trends were observed for higher learning rates. At $lr=0.5$ and $p=20\%$, our method reached 0.010179, outperforming all other configurations, including the Top-5 gradient descent methods and vanilla Adam.

This result demonstrates the robustness of our

method in preserving predictive accuracy, even when a significant portion of the data is corrupted by outliers.

Weight Error (Table 11)

In terms of parameter recovery, our method also outperformed all baselines in most settings. Notably, at $lr=0.1$ and $p=20\%$, our approach reduced the weight error to 0.033661, compared to 0.287513 for standard linear regression and 0.282875 for the Top-5 GD method. This corresponds to an $8.5\times$ improvement over the baseline. While some competing methods showed competitive performance in isolated cases (e.g., Top5 GD at $p=10\%$), none maintained consistent robustness across all tested configurations. Our method exhibited both low estimation and weight errors, validating the advantage of periodically filtering out high-residual samples during training.

Conclusion

The proposed outlier aware training procedure significantly enhances both prediction accuracy and parameter estimation across a variety of learning conditions. By combining full-batch Adam with periodic residual-based filtering, the method is able to effectively mitigate the impact of corrupted samples without introducing instability. The consistent performance gains across multiple settings support the practicality of this approach for regression tasks under noisy or heterogeneous data distributions.

Figure 26. Outlier ratio = 10%

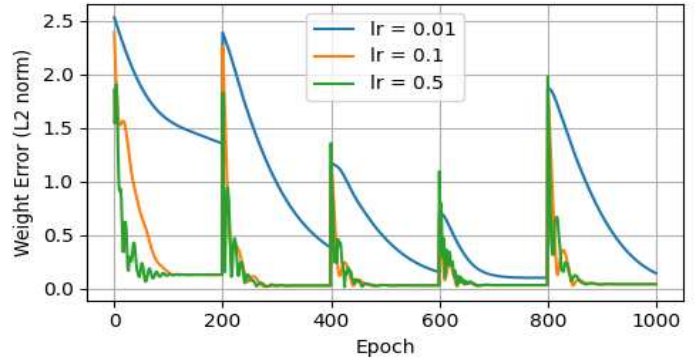
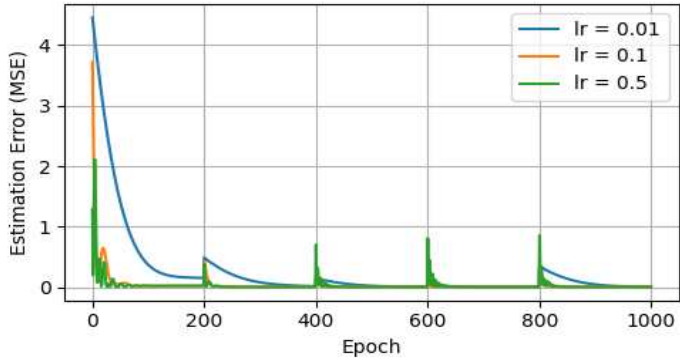


Figure 27. Outlier ratio = 20%

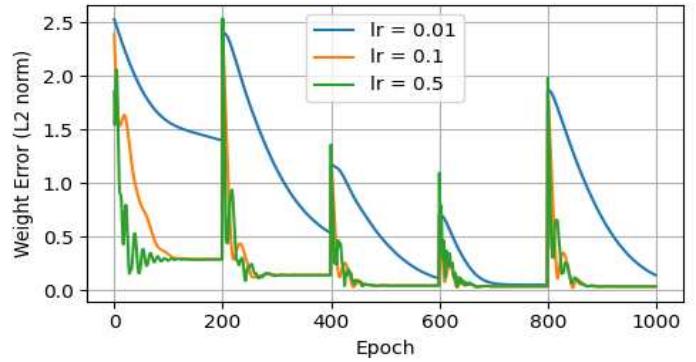
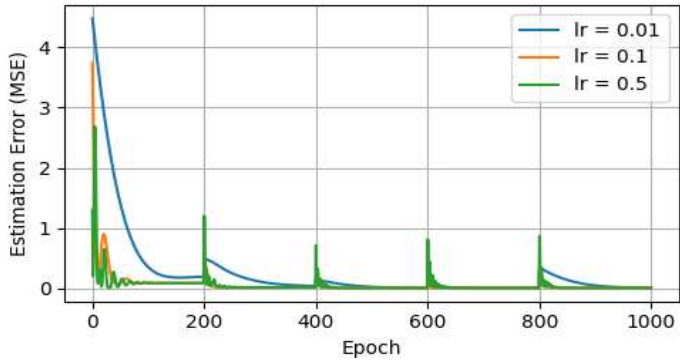
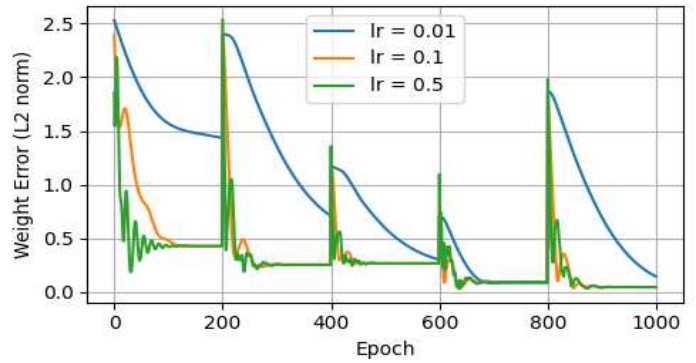
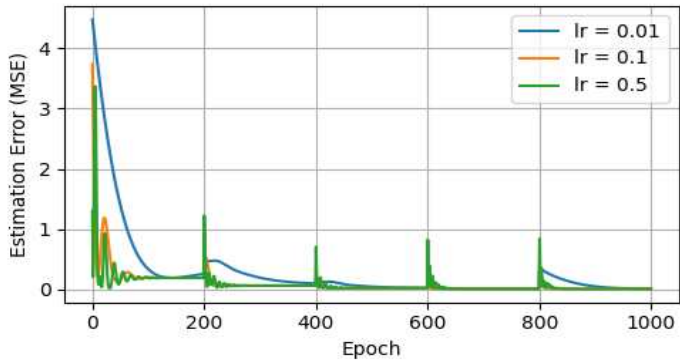


Figure 28. Outlier ratio = 30%



Analysis of Figure: Estimation and Weight Error under Varying Outlier Ratios

The figure illustrates the convergence behavior of both estimation error and weight error as a function of epoch under three different outlier ratios: 10%, 20%, and 30%. Each subplot compares the effect of varying learning rates (0.01, 0.1, 0.5) using the proposed method with periodic outlier removal.

Estimation Error (Left Column)

Across all outlier ratios, the lr=0.1 yields the most fastest convergence to a low estimation error. A higher lr=0.5 leads to rapid convergence initially, but introduces sharp oscillations, particularly following each reset interval, likely due to aggressive updates

after outlier removal. In contrast, lr=0.01 converges smoothly but more slowly, maintaining a higher error level compared to the other settings.

Weight Error (Right Column)

Similar trends are observed for weight error. lr=0.1 maintains a good balance between convergence speed and stability across all outlier ratios. lr=0.5 consistently shows spiky behavior after resets, indicating instability in weight updates despite achieving lower error minima. As the outlier ratio increases, weight error resets become more pronounced, especially at higher learning rates, reflecting stronger shifts in the training set composition.